
Dezentrale marktbasierte Multi-Roboter-Aufgabenzuweisung

Technischer Zwischenbericht - Stufe 1

Tier-0-Simulation, Bidding-Protokoll und empirischer Baseline-Vergleich

Dokumentstatus: Arbeitsfähige Basis für die nächste Projektstufe.

Kernstand: Dezentrale Auktion implementiert, Simulation und Replays verfügbar, Parametersuche und getrennte Holdout-Auswertung abgeschlossen, 18 Tests erfolgreich.

Projektphase:	Stufe 1 / Tier 0
Dokumenttyp:	Technische Zusammenfassung und Übergabebasis
Stand:	11. August 2026
Implementierung:	Python 3.11 oder neuer

Inhaltsverzeichnis

1	Kurzfassung	1
2	Zielsetzung und Abgrenzung	1
3	Systemarchitektur	1
3.1	Schichten und Portabilität	1
3.2	Dezentraler Auktionsablauf	2
3.3	Warum kein zentraler Allocator vorliegt	2
4	Bidding, Priorität und Preemption	2
4.1	Gebotskosten	2
4.2	Zeitabhängige Priorität	3
4.3	Gewählte Parameter	3
5	Tier-0-Simulation	4
5.1	Hauptkonfiguration	4
5.2	Aufgabentypen	4
6	Baselines und fairer Vergleich	4
7	Parametersuche und Nachweismethodik	5
7.1	Trennung von Auswahl und Auswertung	5
7.2	Operationaler Score	5
7.3	Statistik	5
8	Ergebnisse	5
8.1	Holdout-Mittelwerte	5
8.2	Wichtigste gepaarte Effekte	6
8.3	Trade-offs und korrekte Interpretation	6
9	Logging, Replays und Tests	7
10	Bedienung und Reproduktion	7
10.1	Installation und Tests	8
10.2	Empfohlener Lauf mit interaktivem Replay	8
10.3	Vollständige Auswertung	8
11	Grenzen des aktuellen Nachweises	8
12	Empfohlener nächster Schritt: Webots und Pi-Pucks	8
13	Projektartefakte	9
14	Fazit	9

1 Kurzfassung

Die erste Projektstufe ist als lauffähiger, reproduzierbarer Prototyp umgesetzt. Die Roboter verteilen Aufgaben über ein dezentrales Peer-to-Peer-Auktionsverfahren. Ein Roboter entdeckt oder erzeugt eine Aufgabe, kündigt sie mit **ANNOUNCE** an, erhält lokale Gebote über **BID** und vergibt sie mit **AWARD**. Es existiert kein zentraler Allocator, der globale Zustände sammelt oder Gewinner auswählt.

Die Logik wird in einer schnellen zweidimensionalen Tier-0-Simulation in einer $2\text{ m} \times 2\text{ m}$ -Arena geprüft. Simuliert werden sieben Roboter, Bewegung, Startverzögerung, Akkuverbrauch, Funklatenzen, Paketverlust, Erkennung und die Aufgabentypen **GUARD**, **PUSH**, **SURROUND** und **SURVEY**.

Die Marktstrategie wird unter identischen physischen und stochastischen Bedingungen mit Nearest Greedy, Random Assignment und einer Greedy-Ablation ohne **SURVEY** verglichen. Auf 40 getrennten Holdout-Seeds erreicht Market einen operationalen Score von **0,861**, gegenüber **0,831** bei Nearest Greedy und **0,813** bei Random Assignment. Gegen Nearest Greedy ist Market im Mittel rund **10,08 Sekunden schneller** und verteilt die Arbeitslast gleichmäßiger.

Zur qualitativen Prüfung existiert ein interaktiver 2D-Replay-Player mit Start/Pause, Zeitleiste, Einzelschritten, einstellbarer Geschwindigkeit, Fahrspuren und sichtbaren Robotern sowie Aufgaben. Die vollständigen Logs, statistischen Ergebnisse, Diagramme, Parameter und 18 automatischen Tests sind Teil des Projektordners.

Zentrale Aussage: Für die dokumentierte Tier-0-Szenariofamilie liefert die dezentrale Marktstrategie einen statistisch abgesicherten höheren operationalen Gesamtscore als Nearest Greedy und Random Assignment. Dies ist ein reproduzierbarer empirischer Nachweis, aber noch kein universeller mathematischer Dominanzbeweis und noch kein Nachweis für Webots oder reale Pi-Pucks.

2 Zielsetzung und Abgrenzung

Ziel der ersten Stufe ist eine belastbare Software- und Versuchsbasis für dezentrale Multi-Roboter-Aufgabenzuweisung (MRTA). Dazu gehören:

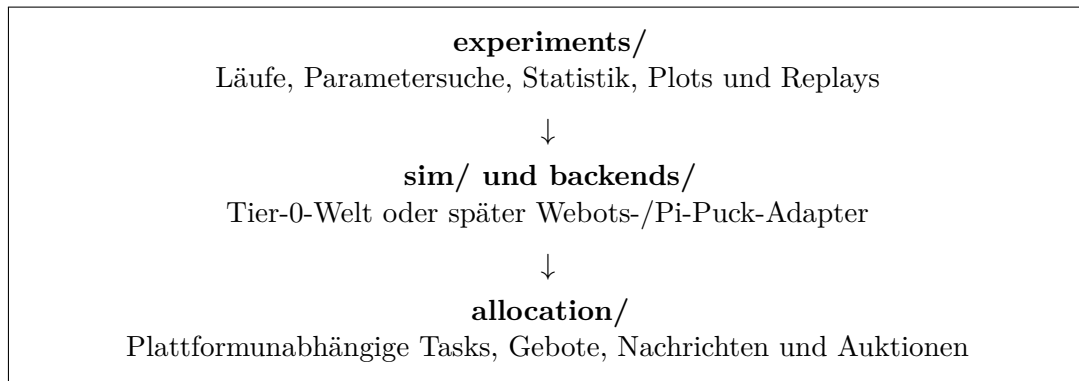
- eine plattformunabhängige, dezentrale Zuweisungslogik,
- ein schnelles Simulationsmodell für viele reproduzierbare Versuche,
- faire Baselines mit identischen Szenarien und Zufallsseeds,
- eine nachvollziehbare Parameterwahl ohne Vermischung von Training und Nachweis,
- portable Logs, statistische Auswertung und visuelle Replays,
- eine klar definierte Schnittstelle für Webots und reale Pi-Puck-Roboter.

Die aktuelle Stufe modelliert Roboter als Punktroboter in einer ebenen Arena. Sie ist bewusst schneller und einfacher als eine physiknahe Webots-Simulation. Reale Differentialantriebsdynamik, Sensorrauschen, Kollisionen, Lokalisierungsfehler und echte Netzwerkeffekte müssen in der nächsten Stufe kalibriert beziehungsweise ergänzt werden.

3 Systemarchitektur

3.1 Schichten und Portabilität

Die Architektur trennt die eigentliche Allokationsentscheidung von Simulation, Hardware und Auswertung:



Das Paket **allocation/** verwendet nur die Python-Standardbibliothek und importiert weder Simulation noch Metriken. Die physische Plattform wird über acht kleine Backend-Fähigkeiten angebunden: Pose lesen, Akku lesen, Ziel anfahren, Zielerreichung prüfen, Nachricht senden, Nachrichten empfangen, RoIs erkennen und Zeit lesen. Dadurch sollen Webots und Pi-Pucks später dieselbe Allokationslogik unverändert nutzen.

3.2 Dezentraler Auktionsablauf

1. Ein Roboter erkennt eine Region of Interest (RoI) oder erzeugt eine fällige **SURVEY**-Aufgabe und wird für genau diese Aufgabe Auktioneer.
2. Er sendet **ANNOUNCE** mit Aufgabenart, Position, benötigter Teamgröße, Prioritätsfunktion und Gebotsdeadline.
3. Jeder erreichbare Roboter berechnet ein Gebot ausschließlich aus seinem lokalen Zustand und sendet **BID**.
4. Der lokale Auktioneer sortiert die Paare aus Kosten und Roboter-ID und sendet **AWARD** an die günstigsten n_{req} Roboter.
5. Die Gewinner navigieren zum Ziel. Eine Mehrroboteraufgabe startet erst, wenn die exakt benötigte Zahl an Gewinnern angekommen ist.
6. Abschluss und Rückgabe werden über **DONE** oder **RELEASE** verbreitet. Bei einer Rückgabe kann der ursprüngliche Auktioneer erneut bieten lassen.

Die fünf Nachrichtentypen **ANNOUNCE**, **BID**, **AWARD**, **DONE** und **RELEASE** sind kompakt und deterministisch als JSON-Bytes serialisiert. Funklatenz und Paketverlust werden unabhängig pro Empfänger simuliert.

3.3 Warum kein zentraler Allocator vorliegt

Jede Roboterinstanz führt ihren lokalen Aufgabenbestand, lokale Gebote, laufende Aufträge und selbst gehostete Auktionen. Die Welt erzeugt Ereignisse und simuliert Bewegung, Funk sowie physische Ausführung, wählt aber keine Auktionsgewinner. Es gibt somit keine globale Instanz, die alle Roboterzustände für eine zentrale Entscheidung zusammenführt.

4 Bidding, Priorität und Preemption

4.1 Gebotskosten

Das niedrigste Gebot gewinnt. Die drei Kostenterme sind auf $[0, 1]$ normiert:

$$c_i(\text{task}) = \alpha \hat{d}_i + \beta \hat{w}_i + \gamma (1 - \hat{e}_i). \quad (1)$$

Dabei bezeichnet $\hat{d}_i$ die geschätzte Distanz, $\hat{w}_i$ die aktuelle Last und $\hat{e}_i$ die verbleibende Energie von Roboter i . Die Priorität steht nicht direkt im Gebot, weil sie innerhalb einer einzelnen Auktion alle Gebote nur um dieselbe Konstante verschieben würde. Stattdessen steuert sie den Wettbewerb zwischen bereits laufenden und neu eintreffenden Aufgaben.

4.2 Zeitabhängige Priorität

Die Priorität altert nach

$$p(\Delta t) = p_0 + (1 - p_0) \left(1 - \exp \left[-(\lambda \Delta t)^k \right] \right). \quad (2)$$

Eine Preemption ist nur während NAVIGATE, niemals während WAIT_PEERS oder EXECUTE, zulässig. Sie wird ausgelöst, wenn

$$\delta (p_{\text{new}} - p_{\text{current}}) - (c_{\text{new}} - c_{\text{current}}) > \theta_{\text{hyst}}. \quad (3)$$

Zusätzlich begrenzt ein Cooldown von 15 Sekunden unnötiges Wechseln. Dadurch bleiben bereits synchronisierte oder ausgeführte Teamaufgaben stabil.

4.3 Gewählte Parameter

Parameter	Wert	Bedeutung
α	1,0	Gewicht der normalisierten Distanz
β	0,5	Gewicht der aktuellen Arbeitslast
γ	0,0	Gewicht der Batteriereserve im 300-Sekunden-Hauptszenario
δ	1,5	Einfluss des Prioritätsgewinns bei Preemption
θ_{hyst}	0,1	Hysterese gegen unnötige Auftragswechsel

Tabelle 1: Durch die Trainingssuche gewählte Market-Parameter.

Der Wert $\gamma = 0$ ist ein Ergebnis der Suche und kein ausgelassener Term. Im 300-Sekunden-Holdout wird kein Roboter vollständig entladen. Ein stärkerer Batterieterm erzeugt dort zusätzliche Wege, ohne einen Ausfall zu verhindern. Für längere Einsätze bleibt γ im Suchraum und das Batteriemodell ist implementiert.

5 Tier-0-Simulation

5.1 Hauptkonfiguration

Bereich	Standardwert	Modellierung
Arena	2 m × 2 m	zweidimensionale Punktroboterwelt
Roboter	7	heterogene Startposition und zufälliger Startakku
Laufzeit	300 s bei $\Delta t = 0,1$ s	schnelle deterministische Zeitschritte
Bewegung	0,08 m/s	effektive Geschwindigkeit plus 1,2 s Startverzögerung
Ereignisse	28 RoIs	feste Hotspots, für alle Strategien identisch
Erkennung	Radius 0,30 m	RoI wird bei räumlicher Nähe erkannt
Funk	5–40 ms, 2 % Verlust	unabhängige Zustellung pro Empfänger
Akku	0,26–1,00 initial	Idle- und wegabhängige Entladung, Depletion-Schwelle 0,05
Coverage	0,05-m-Zellen	SURVEY-Zellen von 0,4 m und zeitabhängige Idleness

Tabelle 2: Zentrale Standardwerte aus `parameters/default.json`.

5.2 Aufgabentypen

Typ	Roboter	Dauer	Rolle im Szenario
GUARD	1	15 s	Einzelroboter sichert eine erkannte RoI
PUSH	2	20 s	koordinierte Zwei-Roboter-Aufgabe
SURROUND	3	10 s	koordinierte Drei-Roboter-Aufgabe
SURVEY	1	variabel	räumliche Abdeckung und erneute Erkundung alter Zellen

Tabelle 3: Unterstützte Aufgabentypen und Teamgrößen.

SURVEY zählt nicht zur normalen Arbeitslast und kann für eine neu erkannte RoI ohne Hysterese freigegeben werden. Mehrroboteraufgaben werden erst physisch ausgeführt, wenn alle benötigten Gewinner am Ziel sind.

6 Baselines und fairer Vergleich

Vier Strategien laufen in derselben Simulations- und Kommunikationsinfrastruktur:

- **Market:** dezentrale Kosten aus Distanz, Last und optional Batterie.
- **Nearest Greedy:** bevorzugt den lokal nächstgelegenen geeigneten Roboter.
- **Random Assignment:** weist unter geeigneten Robotern zufällig zu.
- **Nearest Greedy ohne SURVEY:** Ablation zur getrennten Bewertung des Erkundungsverhaltens und seiner Kommunikationskosten.

Szenario, Funk und Random-Baseline besitzen getrennte, aus dem Lauf-Seed abgeleitete Zufallsgeneratoren. Derselbe Seed erzeugt damit für alle Strategien dasselbe physische Szenario. Ein identischer Lauf ist deterministisch und erzeugt byte-identische Ereignislogs. Die Strategien unterscheiden sich in der Zuweisungsentscheidung, nicht in der zugrunde liegenden Welt.

7 Parametersuche und Nachweismethodik

7.1 Trennung von Auswahl und Auswertung

Die Konfiguration wurde ausschließlich auf den Trainings-Seeds 0 bis 7 ausgewählt. Der anschließende Nachweis verwendet die davon getrennten 40 Holdout-Seeds 1000 bis 1039. Jeder Holdout-Seed wird mit allen vier Strategien gerechnet. So kann pro Seed eine gerichtete gepaarte Differenz gebildet werden, ohne dass unterschiedliche Zufallsfälle den Strategievergleich verzerren.

Die Gittersuche variiert β , γ , δ und θ_{hyst} . Aus 14 nicht-dominierten Pareto-Punkten wird der Punkt mit dem höchsten vorab dokumentierten operationalen Score gewählt.

7.2 Operationaler Score

Komponente	Gewicht	Komponente	Gewicht
Completion Rate	0,30	Detection Rate	0,10
Bearbeitungszeit	0,10	Lastbalance	0,15
Coverage	0,05	mittlere Idleness	0,08
Detektionslatenz	0,07	Restakku	0,05
keine Depletion	0,10		

Tabelle 4: Vorab dokumentierte Gewichte des operationalen Scores.

Alle Komponenten werden vor der Gewichtung auf $[0, 1]$ begrenzt. Der Score dient der Gesamtbewertung und Parameterwahl, ersetzt aber nicht die einzeln berichteten Metriken.

7.3 Statistik

Für jede Baseline werden Mittelwert der gepaarten Differenzen, ein nichtparametrisches 95 %-Bootstrap-Konfidenzintervall mit 20.000 Resamples, ein gepaarter Vorzeichen-Randomisierungstest mit 20.000 Ziehungen, die Win-Rate über 40 Seeds und Holm-korrigierte p-Werte berichtet. Ein Vorteil wird nur dann als solcher bezeichnet, wenn das Konfidenzintervall vollständig positiv ist und $p_{Holm} < 0,05$ gilt.

8 Ergebnisse

8.1 Holdout-Mittelwerte

Strategie	Score	Completion	Zeit [s]	Load-SD	Max. Idle [s]	Msg/Task
Market	0,861	0,918	37,76	1,42	264,99	40,26
Nearest Greedy	0,831	0,887	47,84	1,84	267,09	38,28
Greedy ohne SURVEY	0,813	0,861	46,98	2,09	238,70	10,70
Random Assignment	0,813	0,788	59,60	1,35	284,29	43,72

Tabelle 5: Mittelwerte auf den 40 Holdout-Seeds. Kleinere Zeiten, Load-SD und Idleness sind besser; höhere Scores und Completion Rates sind besser.

8.2 Wichtigste gepaarte Effekte

Gegen Nearest Greedy beträgt der Score-Vorteil von Market $+0,0305$ mit einem 95 %-Bootstrap-Konfidenzintervall von $[+0,0151; +0,0462]$, einer Win-Rate von 77,5 % und $p_{Holm} = 0,0040$. Die mittlere Bearbeitungszeit verbessert sich um 10,08 Sekunden; das Intervall $[6,46; 13,72]$ liegt vollständig über null und die Win-Rate beträgt 87,5 %. Die Laststreuung sinkt um 0,42 Aufgaben.

Gegen Random Assignment steigt der Score im Mittel um 0,0479 und die Completion Rate um 0,1295. Die Bearbeitungszeit verbessert sich um 21,85 Sekunden. Alle drei Effekte sind im gepaarten Holdout signifikant. Damit zeigt die Marktstrategie besonders bei koordinierter Erledigung und zeitlicher Effizienz einen klaren Vorteil.

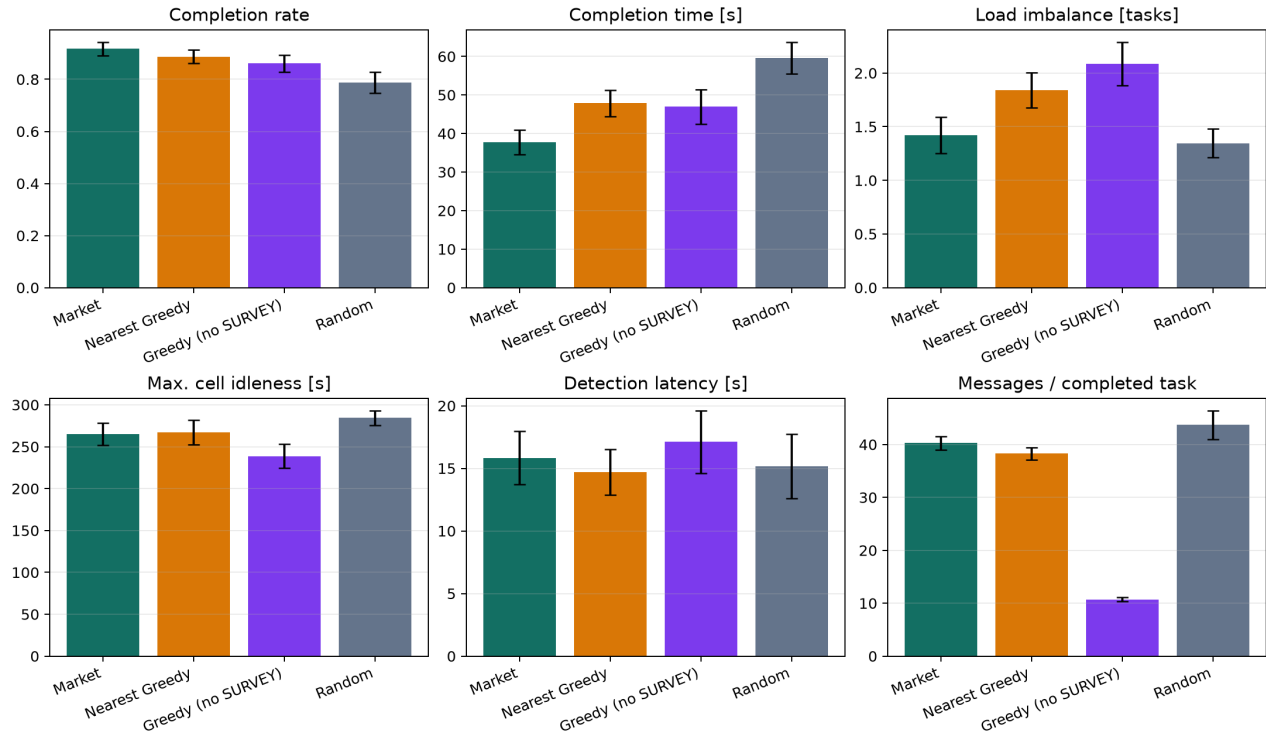


Abbildung 1: Ausgewählte Holdout-Metriken. Die Fehlerbalken zeigen die Streuung der Strategieergebnisse über die ausgewerteten Szenarien.

8.3 Trade-offs und korrekte Interpretation

Market gewinnt nicht zwangsläufig jede Einzelmessung. Gegen Nearest Greedy werden im Mittel rund zwei zusätzliche Nachrichten pro abgeschlossener Aufgabe benötigt; nach Mehrfachkorrektur ist dieser Unterschied knapp nicht signifikant. Der mittlere Restakku ist geringfügig niedriger. Random Assignment kann zufällig eine sehr gleichmäßige Last erzeugen, erledigt aber deutlich weniger Aufgaben und benötigt länger.

Greedy ohne SURVEY erzeugt erwartungsgemäß wesentlich weniger Nachrichten und hat in diesem Boustrophedon-Szenario eine geringere maximale Idleness. Dafür verliert die Ablation bei Completion, Bearbeitungszeit und Lastbalance. Diese Preise werden offen berichtet und nicht durch den Gesamtscore verdeckt.

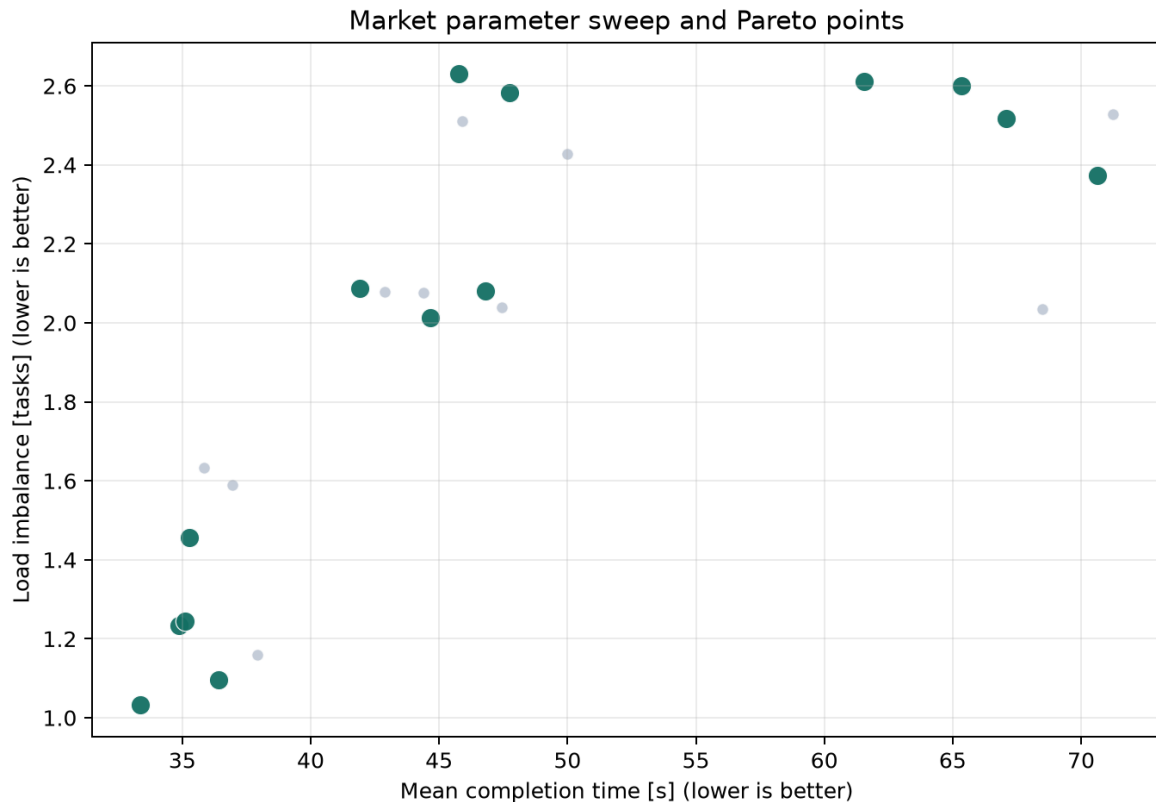


Abbildung 2: Parametersuche: mittlere Bearbeitungszeit gegen Lastungleichgewicht. Dunkle Punkte markieren nicht-dominierte Parameterkonfigurationen.

9 Logging, Replays und Tests

Jeder Lauf schreibt portable Dateien in den gewählten Logordner:

- `events.csv`: Auktionen, Gebote, Zuweisungen, Ausführung und Abschluss,
- `state.csv`: Position und Zustand der Roboter über die Zeit,
- `metadata.json`: Seed, Strategie, Parameter und Laufkonfiguration,
- `replay.html`: interaktiver Offline-Player,
- optional `replay.gif`: nicht interaktive Animation.

Der HTML-Player zeigt die Roboterbewegung in der 2D-Arena sowie Aufgaben und Zuweisungen. Play/Pause, Zeitregler, Schritte von einer Sekunde, Geschwindigkeiten von 0,25-fach bis 20-fach, Fahrspuren, SURVEY-Ebenen, Leertaste und Pfeiltasten erlauben eine gezielte visuelle Analyse. Er läuft lokal im Browser ohne Server und ohne Internetverbindung.

Die Testsuite enthält 18 Unit- und Invariantentests. Sie prüft unter anderem Nachrichtenformate, Gebots- und Auktionslogik, Priorität, Determinismus, Mehrroboter-Synchronisation, Depletion und RELEASE, Paketverlust, plattformunabhängige Imports und die Integrität der Simulationsabläufe.

10 Bedienung und Reproduktion

Der Projektordner kann auf einem beliebigen Computerpfad entpackt werden. Es existiert keine Abhängigkeit von einem persönlichen Windows-Benutzernamen. Benötigt wird Python 3.11 oder

neuer. Alle Befehle werden im Ordner ausgeführt, in dem `README.md` und `pyproject.toml` liegen.

10.1 Installation und Tests

```
python -m pip install -e .
python -m unittest discover -s tests -v
```

10.2 Empfohlener Lauf mit interaktivem Replay

```
python -m experiments.run_once --strategy market --seed 42 \
    --output logs\market_42 --interactive-replay
```

Danach wird `logs/market_42/replay.html` im Browser geöffnet. Für einen fairen visuellen Vergleich werden derselbe Seed und getrennte Ausgabeordner verwendet, während `market` nacheinander durch `nearest_greedy` und `random_assignment` ersetzt wird.

10.3 Vollständige Auswertung

```
python -m experiments.compare
```

Der Befehl reproduziert Parametersuche, Holdout-Läufe, Statistik und Abbildungen. Auf der Entwicklungsmaschine benötigt er ungefähr 2,5 Minuten; die Laufzeit kann auf anderen Computern abweichen. Die bereits berechneten Resultate liegen im Ordner `results/` und müssen zum Lesen oder Präsentieren nicht neu erzeugt werden.

11 Grenzen des aktuellen Nachweises

Die Ergebnisse gelten für die implementierte Tier-0-Szenariofamilie, die dokumentierten Parameter und die 40 Holdout-Seeds. Sie belegen keine universelle Überlegenheit der Marktstrategie und garantieren nicht, dass dieselbe Rangfolge in Webots oder auf realen Robotern bestehen bleibt.

Inbesondere müssen folgende Punkte in späteren Stufen berücksichtigt werden:

- reale Kinematik, Beschleunigung, Drehzeiten und Kollisionsvermeidung,
- Sensorrauschen, Sichtfeld, Fehl- und Mehrfachdetektionen,
- Lokalisierungsfehler und Koordinatentransformationen,
- Reichweite, Verzögerung, Burst-Verluste und Duplikate echter Funknetze,
- reale Akkuentladung unter Fahrt, Stillstand, Sensorik und Kommunikation,
- Roboterenausfälle, blockierte Wege und Recovery-Strategien,
- Skalierung auf andere Arena-, Team- und Aufgabengrößen.

12 Empfohlener nächster Schritt: Webots und Pi-Pucks

Die nächste Stufe sollte die bestehende Allokationslogik nicht neu schreiben, sondern die acht Backend-Fähigkeiten für Webots und anschließend für Pi-Pucks implementieren. Ein sinnvoller Übergang besteht aus vier Schritten:

1. **Schnittstellen einfrieren:** Nachrichtenformat, Koordinatensystem, Zeiteinheit, Robot-IDs, Fehlerzustände und Semantik der acht Backend-Funktionen gemeinsam abstimmen.
2. **Webots-Adapter bauen:** Pose, Batterie, Bewegung, Zielerreichung, Kommunikation, Detektion und Uhrzeit an die vorhandene Schnittstelle anbinden.
3. **Tier 1 kalibrieren:** Insbesondere `effective_speed`, `startup_delay`, Ankunftstoleranz, Funkverlust, Latenz und Batteriemodell durch Messläufe an Webots beziehungsweise Pi-Pucks bestimmen.
4. **Gepaarten Vergleich wiederholen:** Market, Nearest Greedy und Random mit identischen Webots-Seeds und danach identischen Realversuchen vergleichen. Die Tier-0-Analyse dient dabei als Referenz, nicht als vorausgesetztes Ergebnis.

Als unmittelbare Teamentscheidung sollten zunächst die Zielplattform, verfügbare Pi-Puck-Hardware, Webots-Version, Kommunikationsart, RoI-Sensorik und das gemeinsame Koordinatensystem festgelegt werden. Danach kann der Backend-Adapter implementiert und gegen die vorhandenen 18 Tests sowie neue Hardware-in-the-Loop-Tests geprüft werden.

13 Projektartefakte

Pfad	Inhalt
<code>allocation/</code>	Plattformunabhängige Tasks, Gebote, Nachrichten und Auktionen
<code>backends/</code>	Acht-Fähigkeiten-Schnittstelle und Tier-0-Adapter
<code>sim/</code>	Roboter, Arena, Funk, Batterie, Coverage und Ereignisse
<code>metrics/</code>	CSV-Logging und Metrikberechnung
<code>experiments/</code>	Läufe, Parametersuche, Statistik, Plots und Replays
<code>parameters/</code>	Standardparameter und Suchraum
<code>tests/</code>	18 automatisierte Unit- und Invariantentests
<code>logs/example/</code>	Fertiger Beispiel-Log mit GIF und interaktivem HTML-Replay
<code>results/</code>	Vollständige Holdout-Ergebnisse, Tabellen und Abbildungen
<code>docs/</code>	Architektur, Methodik und Anforderungszuordnung
<code>references/</code>	Ursprüngliches Briefing und Projektvorschlag als PDF

14 Fazit

Stufe 1 liefert eine funktionsfähige, dezentrale MRTA-Basis mit klarer Plattforttrennung, reproduzierbarer Simulation, fairen Baselines, statistisch getrenntem Training und Holdout sowie visueller und maschinenlesbarer Nachvollziehbarkeit. Der Marktansatz verbessert in der untersuchten Szenariofamilie den operationalen Score, die Bearbeitungszeit und gegenüber Nearest Greedy auch die Lastverteilung.

Damit ist die Grundlage für den nächsten Schritt vorhanden: die bestehende Logik über einen Webots- und später Pi-Puck-Backend-Adapter auf realistischere Dynamik zu übertragen, Parameter messtechnisch zu kalibrieren und denselben gepaarten Vergleich auf Tier 1 und Hardware zu wiederholen.

Maßgebliche maschinenlesbare Konfiguration: `parameters/default.json`.

Vollständige Auswertung: `results/comparison_summary.md` und `results/paired_effects.csv`.